Miguel Tillero

PLATAFORMA EDUCATIVA · ANEXO TÉCNICO

Debajo *del capó*

Arquitectura, modelo de datos, seguridad por filas, orden de ejecución y las decisiones técnicas con su porqué.

Miguel David Tillero Álvarez
Octubre de 2026 · Versión 1.1
Documento complementario · no indispensable para valorar el proyecto

---

> A QUIÉN VA DIRIGIDO
>
> A quien quiera comprobar que lo descrito en el manual está realmente construido, y cómo. No hace falta leerlo para hacerse una idea del proyecto: para eso está el documento de presentación.

1 · Arquitectura

Dos piezas, deliberadamente pocas:

PIEZA	QUÉ ES	QUÉ RESUELVE
GitHub Pages	Alojamiento de sitio estático	Sirve las 47 páginas. Sin servidor que mantener, sin costo, con publicación automática en cada cambio y con todo el historial versionado.
Supabase	Base de datos PostgreSQL con autenticación y reglas de acceso	Guarda los datos, gestiona las cuentas y decide quién puede leer qué. El navegador habla directamente con ella.

Cómo encaja

```
Navegador (página estática)
      │
      │  biblioteca oficial de Supabase, con la clave pública
      ▼
Supabase · PostgreSQL
      ├── Autenticación  → quién eres
      ├── Seguridad por filas (RLS) → qué puedes ver
      └── Funciones SECURITY DEFINER → lo que necesita privilegios
```

No hay servidor propio en medio. La página llama a la base de datos con la clave pública, y es la base de datos la que decide qué devuelve según quién pregunta. Eso tiene una consecuencia que conviene entender: **toda la seguridad vive en la base**. No hay un intermediario donde esconder reglas; si una regla falta, el dato queda expuesto.

> POR QUÉ ESTA ARQUITECTURA Y NO OTRA
>
> Un proyecto de un solo docente no puede permitirse mantener un servidor. Esta combinación da autenticación, base de datos relacional y permisos finos sin costo fijo ni administración de sistemas, y deja todo el código en un repositorio auditable. El precio es la disciplina que exige: cada tabla nueva necesita su política de acceso el mismo día en que se crea.

2 · Modelo de datos

30 tablas. Agrupadas por función:

Personas y acceso

`profiles`	Perfil de cada cuenta: nombre, teléfono, rol (*docente*, *estudiante*, *admin*, *representante*) y los datos del representante declarados al inscribirse.
`tutorias`	Qué representante puede ver a qué estudiante. Una fila por pareja: un representante puede tener varios hijos, y un estudiante puede tener madre y padre.

Oferta académica

`cursos`	La materia: nombre, nivel, descripción.
`grupos`	La instancia concreta: código, fechas, horario, modalidad, cupo, costo, estado, y las dos fechas de plazo (inscripción y pago).
`horarios`	Días de la semana y horas de cada grupo.
`sesiones`	Cada clase con su fecha, tema y contenido. Es la planificación.
`feriados`	Días sin clase, para que la generación automática los salte.
`grupo_enlaces`	Enlace de la clase y del grupo de mensajería, **en tabla aparte y bajo llave**.
`grupo_progresion`	Histórico de unidades ya trabajadas, para el recorrido del estudiante.
`metodos`, `metodo_unidades`	Catálogo de métodos y sus unidades, para crear cursos más rápido.

Inscripción, asistencia y dinero

`inscripciones`	Quién está en qué curso y en qué estado: *pendiente*, *activa*, *rechazada*, *retirada*, *finalizada*.
`asistencia`	Presente o ausente por sesión y estudiante, con observación.
`pagos`	Monto, moneda, concepto, estado y fecha de cobro.
`egresos`	Gastos, para el balance neto.

Evaluación

`evaluaciones`, `notas`	Evaluaciones del curso y la calificación de cada estudiante, con comentario y desglose por partes.
`examenes`	Catálogo de exámenes en línea: título, código de acceso, duración, si está abierto.
`examenes_claves`	**La clave de respuestas.** Solo accesible para administración; nunca sale al navegador.
`examenes_intentos`	Cada intento: cuándo empezó, qué respondió, cuándo envió, desde dónde.
`boletines`, `boletin_notas`	Boletín por estudiante con su identificador secreto, y sus resultados.
`validaciones_nivel`	Validaciones de nivel alcanzadas.

Otros

`materiales`	Repositorio por grupo, con categoría y unidad.
`eventos`	Eventos del sitio público.
`tipos_constancia`, `solicitudes_constancia`	Preparadas; la emisión aún no está construida.
`vea_carnet`, `vea_estudiantes`	Club «Vendredi entre Amis».

3 · Seguridad por filas

PostgreSQL permite decidir, **fila por fila**, quién puede leerla o escribirla. Es el mecanismo que sostiene toda la privacidad del proyecto: un estudiante consulta la misma tabla que el docente, pero la base le devuelve únicamente lo suyo.

El error que costó una tarde: la recursión

El 11 de septiembre, el primer día de la plataforma, las reglas entraron en **recursión infinita**. La causa es instructiva: para saber si alguien es docente hay que consultar su perfil; pero la tabla de perfiles tiene su propia regla, que para evaluarse necesita saber si es docente. El bucle no termina nunca y la base devuelve error.

La solución, aplicada en todo el proyecto desde entonces, es encapsular esas comprobaciones en funciones con privilegios propios, que no vuelven a pasar por las reglas:

```
create function public.es_admin() returns boolean
  language sql stable security definer
as $$
  select exists (
    select 1 from public.profiles p
     where p.id = auth.uid() and p.role in ('docente','admin')
  );
$$;
```

El mismo día se cerraron **tres huecos** por los que un usuario podía ver datos que no le correspondían.

Dos huecos más, encontrados en el ciclo real

El 25 de septiembre, preparando el primer ciclo, aparecieron dos fallos reales de privacidad, ambos en el mismo sitio:

- Una política permitía a **cualquier visitante anónimo** leer la tabla de grupos, que entonces incluía el enlace de la clase con su contraseña.
- Corregido eso, seguía pudiendo verlo **un inscrito que no había pagado**.

La corrección fue estructural, no un parche: los enlaces se mudaron a `grupo_enlaces`, una tabla aparte cuya regla de lectura exige inscripción *activa*. Se comprobó cada caso: visitante anónimo, registrado sin inscripción, inscrito pendiente, inscrito activo y docente.

Un tercer episodio: lo que estaba escrito en el repositorio

El 9 de octubre se corrigió un fallo de la misma familia, pero de origen distinto: **los enlaces de la clase estaban escritos dentro de un archivo del repositorio**, que es público. Ninguna regla de acceso protege lo que está publicado como código. Pasaron a la base de datos, donde la regla de lectura exige inscripción activa.

La lección, que vale para cualquier proyecto alojado así: *la seguridad por filas no alcanza a lo que uno mismo publica en el código*. Hay que mirar las dos cosas.

La decisión de diseño más importante del proyecto

Para el acceso de las familias había dos caminos. El primero: escribir políticas de lectura en las seis tablas implicadas —asistencia, notas, pagos, sesiones, cursos, inscripciones—. El segundo: **una sola función** que devuelva el panel ya armado.

Se eligió el segundo, por una razón concreta: seis tablas son seis sitios donde equivocarse, y un error ahí significa que una madre vea las notas del hijo de otra. La función:

- Se ejecuta con privilegios propios, de modo que no depende de políticas repartidas.
- Filtra por el usuario autenticado contra la tabla de tutorías.
- **No acepta ningún parámetro.** No hay nada que manipular desde el navegador para asomarse a otro estudiante.

Se verificó con dos familias simuladas en una base de pruebas: cada representante vio únicamente a su hija, y una estudiante no figuraba como representante de nadie.

Cómo se protege la evaluación en línea

Sentinelle aplica el mismo principio llevado al extremo: **el alumno no tiene acceso a ninguna tabla**. Solo puede invocar tres funciones —iniciar, guardar, enviar— con el identificador de su intento. La clave de respuestas vive en una tabla que únicamente administración puede leer, el reloj lo lleva el servidor, y cada guardado deja registro de su origen.

4 · Orden de ejecución de los guiones

Los 29 archivos de la carpeta `supabase/` tienen dependencias entre sí. Cada uno declara en su cabecera después de cuál debe ejecutarse. Todos son **idempotentes**: volver a ejecutarlos no rompe nada.

Instalación desde cero

ORDEN	ARCHIVO	QUÉ APORTA
1	`schema.sql`	Tablas base, roles y primeras políticas.
2–6	`schema_v2` … `schema_v6`	Evolución del esquema: grupos, pagos, eventos, métodos.
7	`ciclo-a1-oct2026.sql`	El ciclo A1.4: curso, grupo, horarios, doce sesiones y dos evaluaciones.
8	`ciclo-a1-detalles.sql`	Enlaces de clase, unidad actual, mensaje de bienvenida, progresión.
9	`materiales-y-resultados.sql`	Repositorio de materiales y notas devueltas.
10	`representante-y-enlaces.sql`	Datos del representante; los enlaces pasan a tabla propia bajo llave.
11	`plazo-inscripcion.sql`	Plazos de inscripción y de pago, separados, en hora de México.
12	`evaluaciones-sentinelle.sql`	Sistema de exámenes en línea con control de integridad.
13	`resultados-a1-4.sql`	Publicación de los resultados en «Mis resultados».
14	`boletines.sql`	Boletines con enlace privado para las familias.
15	`representantes-acceso.sql`	Cuenta propia del representante y función del panel.
16	`representante-correo.sql`	Corrección del correo del representante y enlace manual.
17	`libro-virtual.sql`	Almacén privado del método, subida por partes y lectura solo para inscripción activa.
18	`ciclo-b2-oct2026.sql`	El ciclo B2: curso, grupo, horarios y diez sesiones planificadas.

`TODO-EN-UNO.sql` agrupa los pasos 1 a 7 en un solo archivo, para una instalación inicial de una sola pegada.

Utilidades · se ejecutan cuando hacen falta

`consultas-del-dia-a-dia.sql`	Siete bloques sueltos: estado del grupo, activar a alguien, registrar un pago, quién es quién, clases que vienen, dinero del ciclo.
`aprobar-inscripciones.sql`	Aprobar inscripciones sin pasar por el panel.
`baja-estudiante.sql`	Retirada de un estudiante a mitad de ciclo.
`estudiantes-sin-grupo.sql`	Asignar grupo a quien se registró sin él.
`evento_*.sql`, `carnet_*.sql`	Eventos del sitio y carnets del club.

Banco de pruebas

La carpeta `supabase/pruebas/` contiene una simulación del entorno que permite ejecutar y verificar los guiones en un PostgreSQL local, sin tocar los datos reales. Todas las funciones descritas en este anexo se probaron ahí antes de aplicarse: los emparejamientos de representantes en ambos sentidos, el aislamiento entre familias, los límites de los plazos y la baja de estudiante.

5 · Decisiones técnicas y su porqué

Sitio estático en lugar de aplicación con servidor

Sin servidor no hay servidor que caiga, que actualizar ni que pagar. El precio es que toda la lógica sensible debe vivir en la base de datos. Para un proyecto de un docente, el intercambio compensa con holgura.

Una función en lugar de políticas repartidas, para el panel de familias

Menos superficie de error. Seis políticas en seis tablas son seis oportunidades de dejar un hueco; una función sin parámetros, filtrada por el usuario autenticado, es una sola cosa que revisar. Ya habíamos pagado el precio de una recursión de políticas; no tenía sentido repetir el patrón.

Boletín con identificador secreto en lugar de cuenta obligatoria

Las notas no podían esperar a que cada familia creara una cuenta. El identificador es imposible de adivinar y no permite listar los demás boletines. La contrapartida es explícita: quien reciba el enlace, entra. Por eso se envía siempre por mensaje privado.

Fechas en hora de Ciudad de México, no en la hora universal

Detalle pequeño con consecuencia grande. PostgreSQL trabaja por omisión en hora universal; con esa referencia, el martes a las seis de la tarde en México ya es miércoles, y el plazo de inscripción se habría cerrado **una hora antes de la clase**. Todas las comparaciones de fecha pasan por una función que fija la zona horaria.

Pagos manuales en lugar de plataforma de cobro

Se construyó una integración de cobro automático y **se retiró** una semana después. No correspondía a la forma real de trabajar: transferencias y depósitos sin emisión de factura con RFC. Mantener una función que no refleja la realidad del negocio es deuda, no capacidad.

Sesión que sobrevive a un navegador defectuoso

Seis intentos de arreglar el inicio de sesión apuntaban al código; la causa estaba en el **almacenamiento del navegador**, que a veces no guarda lo que se le pide. La solución fue comprobar cada escritura leyéndola de vuelta y, si falla, degradar a almacenamiento de sesión o de memoria. Y, sobre todo, no borrar jamás una sesión que todavía es válida: una limpieza demasiado agresiva puede dejar a alguien fuera de su cuenta permanentemente.

El libro del curso fuera del repositorio

El método que los alumnos compran no podía guardarse junto al código: el repositorio es público, y cualquiera con la dirección descargaría el PDF. Va a un almacén privado, y cada lectura usa un enlace firmado que caduca. Como el plan gratuito limita el tamaño de cada archivo, **la página de carga parte el PDF en trozos en el navegador del docente** antes de subirlo: una restricción de la herramienta convertida en una función con barra de avance.

Correo desde una cuenta personal en lugar de un servicio profesional

Se evaluó un proveedor transaccional con dominio propio. Requiere un dominio registrado, que el proyecto no tiene. La alternativa —enviar desde una cuenta de correo personal con contraseña de aplicación— es menos elegante pero **entrega mejor hoy**: una dirección con años de historial inspira más confianza a los filtros de correo que un dominio estrenado ayer. El camino al escalón siguiente queda documentado.

Todo en español, también el código

Nombres de tablas, funciones, variables y comentarios en español. El proyecto lo mantiene un profesor de francés que trabaja en español, no un equipo de desarrollo angloparlante. Que el código se lea en el idioma en que se piensa reduce el coste de volver a él tres meses después.

6 · Deuda técnica reconocida

Lo que un revisor técnico encontraría, dicho antes de que lo encuentre:

ASUNTO	SITUACIÓN
Esquema en seis versiones sucesivas	De `schema.sql` a `schema_v6.sql`. Funciona y es idempotente, pero un esquema consolidado sería más fácil de leer para quien llegue nuevo.
Operaciones que solo existen en SQL	Baja de estudiante, plazos y enlace manual de representantes se ejecutan desde el editor de la base, no desde el panel. Funciona, pero exige al docente pegar consultas.
Estilos repetidos entre páginas	Cada panel lleva sus propios estilos incrustados. Se unificó el bloque de audio, que estaba copiado en once páginas; queda trabajo similar pendiente.
Pruebas automatizadas parciales	Hay banco de pruebas para la base de datos y pruebas de navegador para funciones concretas, pero no una batería que se ejecute en cada cambio.
Dos cuentas de personal	Una con rol *admin* y otra con rol *docente*, ambas del mismo profesor. Funciona, pero se presta a confusión y conviene unificar.

Vive el francés · Plataforma educativa
Miguel David Tillero Álvarez · Profesor de francés (FLE)
Anexo técnico · Versión 1.1 · Octubre de 2026